Manufacturing Equipment Copilot: Backend and Agent Guide

This document explains the project for a beginner. It describes what happens from the moment a user types a question until the answer appears, why each backend file exists, how BigQuery retrieval works, and how documents become searchable.

1. What the application does

The application is a multimodal retrieval-augmented generation (RAG) assistant for manufacturing manuals. It can answer questions using indexed PDF manuals, return grounded text and tables, show manual images with descriptions, and include relevant manual video or QR links.

The system has four major parts:

1. Frontend: the React interface where the user starts a chat, uploads an image, and reads the response.
2. FastAPI backend: validates requests, manages sessions, runs retrieval, calls the agent, and returns JSON.
3. BigQuery and Cloud Storage: BigQuery stores searchable chunks and related document metadata; Cloud Storage stores processed manual assets.
4. Google models and Document AI: Gemini creates query/document embeddings and generates the final grounded response; Document AI is used offline to inspect QR codes during ingestion.

2. Repository structure

text
Multimodel-RAG-for-Manufacturing-Industry/
??? backend/
?   ??? app/
?   ?   ??? main.py FastAPI application and router setup
?   ?   ??? config.py Environment configuration and validation
?   ?   ??? api/ HTTP endpoints
?   ?   ??? agent/ Planning, Gemini agent, orchestration
?   ?   ??? retrieval/ Embeddings and BigQuery retrieval
?   ?   ??? media/ Images, PDFs, videos, and asset loading
?   ?   ??? responses/ Response schemas and response assembly
?   ?   ??? sessions/ In-memory chat sessions
?   ??? scripts/ Offline QR and connection utilities
?   ??? tests/ Offline backend tests
?   ??? .env Local secrets and settings (never commit)
?   ??? .env.example Safe configuration template
?   ??? requirements.txt Backend dependencies
?   ??? run.cmd Backend launcher
??? frontend/ React/Vite application
??? src/simple_ingestion/ PDF processing and BigQuery loading
??? data/ Local raw/processed data when used
??? terraform/ Infrastructure definitions

3. Starting the application

Start the backend from backend:

powershell
.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload

The important detail is app.main:app. The old main:app path is no longer correct because the FastAPI file is inside backend/app.

Start the frontend from frontend:

powershell
npm.cmd run dev

The normal URLs are:

text
Frontend: http://localhost:5173
Backend: http://127.0.0.1:8000
Health: http://127.0.0.1:8000/health

4. Configuration

backend/app/config.py loads backend/.env, validates required settings, and exposes constants used by the rest of the backend. The important settings are:

Setting	Purpose
GOOGLE_CLOUD_PROJECT	Google Cloud project used by Gemini, BigQuery, and storage
BQ_DATASET	BigQuery dataset, currently manufacturing_rag
BQ_LOCATION	BigQuery job location
EMBEDDING_MODEL	Query/document embedding model, currently gemini-embedding-001
EMBEDDING_DIMENSIONS	Shared vector size, currently 768
GCS_BUCKET_NAME	Cloud Storage bucket containing manual assets
REQUEST_TIMEOUT_SECONDS	Maximum time for one chat request
FRONTEND_ORIGIN	Allowed browser origin for CORS

There is deliberately no active single-machine setting. Retrieval reads machine_name from BigQuery, so HSG-2550, HAB-3232, and future machines can coexist.

5. Complete request flow

text
User types question
        ?

?
Frontend sends multipart POST /chat
?
?
chat_routes.py validates session, text, and optional image
?
?
chat_service.py checks inventory requests and calls planner
?
?
planner.py creates a QueryPlan with topic, query, and media needs
?
?
embeddings.py creates a 768-value RETRIEVAL_QUERY vector
?
?
manual_search.py searches document_chunks in BigQuery
?
?
selected chunks are enriched with images, links, tables, sections
?
?
prepare_media.py loads approved assets from Cloud Storage
?
?
agent.py receives only approved evidence and allowed media
?
?
Gemini returns structured AgentAnswer JSON
?
?
response_builder.py validates evidence and assembles final blocks
?
?
FastAPI returns JSON; frontend renders text, tables, images, and videos

6. HTTP API files

backend/app/main.py

Creates the FastAPI application, configures CORS, runs startup checks, and includes the four routers. It is the ASGI entry point used by Uvicorn.

backend/app/api/chat_routes.py

POST /chat is the main endpoint. It:

- loads the session;

- validates a maximum question length;
- accepts PNG, JPEG, or WebP uploads;
- limits uploads to 5 MB;
- normalizes uploaded images;
- calls answer_question with a request timeout;
- records the user and assistant messages;
- converts authentication, model, timeout, and validation failures into useful HTTP errors.

backend/app/api/session_routes.py

Provides POST /sessions, GET /sessions/{id}, and DELETE /sessions/{id}. The frontend creates a session before sending chat messages and uses that ID for the conversation.

backend/app/api/image_routes.py

GET /images/{asset_id} serves an approved image asset when the asset identifier is valid. Images are served through this route rather than exposing arbitrary filesystem paths.

backend/app/api/health_routes.py

GET /health confirms that FastAPI is running and reports the session implementation. It is a process check, not a full BigQuery/Gemini connectivity test.

7. Agent and orchestration files

backend/app/agent/planner.py

The planner makes a small preliminary Gemini call and returns a QueryPlan. It identifies the search query, main topic, follow-up state, whether images/videos/sources are wanted, and whether clarification is needed. It does not answer the technical question.

backend/app/agent/instructions.py

Contains the system rules given to the final agent. The rules require answers to use approved manual evidence, prohibit invented procedures, require evidence IDs for technical blocks, and require image descriptions when images are shown.

backend/app/agent/agent.py

Defines the Google ADK root agent and its model settings. It receives a JSON payload containing the question, workflow state, approved evidence, images, links, and warnings. It returns structured JSON rather than free-form text.

backend/app/agent/chat_service.py

This is the orchestration layer. It connects planning, retrieval, media preparation, ADK execution, validation, and session state.

For inventory questions such as ?which machine documents do you have??, it uses list_available_documents() directly so every distinct machine/document pair is returned. It does not use similarity ranking for that request.

8. Retrieval and BigQuery

backend/app/retrieval/embeddings.py

Creates a cached Vertex AI client and calls gemini-embedding-001 with task type RETRIEVAL_QUERY. The output must contain exactly 768 numbers. A different vector length raises an error because cosine distance requires equal-length vectors.

backend/app/retrieval/bigquery_client.py

Creates a cached BigQuery client, binds query parameters, limits billed bytes, waits for the query, and converts BigQuery rows into normal Python dictionaries. It caches the client connection, not query results.

backend/app/retrieval/manual_search.py

This is the core retriever.

1. It embeds the user question.
2. It extracts up to 12 simple keyword terms.
3. It searches only document_chunks for the initial candidates.
4. It compares the question vector with stored vectors using ML.DISTANCE(..., 'COSINE').
5. It adds a small keyword bonus to the distance score.
6. It keeps the best 10 semantic seed chunks, plus limited media candidates when requested.
7. It expands the selected chunks to the same document/page.
8. It enriches those selected pages from geometry, images, table rows, links, relationships, and sections.
9. It selects up to 24 text chunks, up to 6 image rows, and up to 8 link rows within the response budget.
10. It loads saved QR links for each machine represented in the selected chunks.

The initial retrieval source is always document_chunks. Other tables add context only after a chunk has been selected.

The current vector constraints are intentional:

text
embedding_model gemini-embedding-001
embedding_dimensions 768

Adding more manuals increases the number of rows, not the vector dimension. If a document has another dimension, it must be re-embedded to 768 before loading.

list_available_documents()

This function runs a simple grouped query over document_chunks:

sql

```
SELECT machine_name, document_name
FROM document_chunks
GROUP BY machine_name, document_name
ORDER BY machine_name, document_name
```

It is used for inventory questions and is why the agent can list all machines instead of returning only the nearest HSG chunks.

backend/app/retrieval/qr_link_search.py

Reads verified QR destinations saved in the QR links table. It filters by machine, document, and page so unrelated QR links are not shown.

9. Media and evidence

backend/app/media/prepare_media.py

Takes retrieved chunks and prepares the media payload for the agent. It ensures every supplied image/link retains its source chunk and records warnings instead of silently losing an asset.

backend/app/media/manual_images.py

Loads images from Cloud Storage, renders PDF pages when needed, normalizes uploaded images, and uses bounded caches for repeated assets. It never allows arbitrary user paths.

backend/app/media/video_links.py

Validates URLs and identifies approved manual video targets. Links from unrelated pages are excluded. YouTube links are normalized for frontend playback.

10. Structured response files

backend/app/responses/schemas.py

Defines Pydantic models such as AgentAnswer, AnswerBlock, and ImageDescription. These schemas force the model to return predictable JSON.

backend/app/responses/response_builder.py

Validates evidence IDs, rejects unsupported or invented evidence, preserves all relevant selected media, and converts the model answer into frontend blocks. Image blocks include a description, and source blocks identify the manual, page, and chunk.

11. Sessions

backend/app/sessions/session_store.py

Uses the ADK in-memory session service. A session stores recent messages and workflow state such as the main topic, guided troubleshooting step, and completion status. Sessions expire after the configured TTL, currently up to one hour. Restarting the backend clears in-memory sessions.

12. Ingestion flow

The ingestion code under src/simple_ingestion runs before chat retrieval:

1. pdf_extractor.py extracts PDF text and page elements.
2. table_extractor.py extracts table content.
3. chunk_builder.py creates text, image, table, and link chunks with machine/document/page metadata.
4. relationship_builder.py records relationships between page elements.
5. embedding_generator.py creates RETRIEVAL_DOCUMENT embeddings using 768 dimensions.
6. gcs_ingestor.py uploads processed assets to Cloud Storage.
7. bigquery_loader.py validates vectors and loads the normalized tables into BigQuery.
8. The loader rejects a vector whose declared or actual dimension differs from the shared 768 dimension.
9. pipeline.py coordinates these steps; main.py provides the ingestion CLI.

The machine name is metadata supplied during ingestion. It may be different for every manual, but every document must use the same embedding model and dimension for shared vector search.

13. Document AI and QR workflow

backend/scripts/extract_qr_links.py

Runs offline Document AI processing for a PDF page or image, extracts QR results, validates safe URLs, and writes a reviewable JSON result. It does not modify BigQuery.

backend/scripts/save_qr_links.py

Saves only manually verified QR results to the QR links table. Chat reads these saved links; chat does not call Document AI or write BigQuery data.

backend/scripts/check_connections.py

Performs a small connectivity check for configured Google services.

14. Error handling

- 400: invalid or empty request, unsupported image type, or oversized upload.
- 404: missing session or asset.
- 410: expired or deleted session.
- 502: model returned invalid structured evidence.
- 503: Google authentication, Gemini, BigQuery, or asset service failure.
- 504: request exceeded the backend timeout.

If chat reports service-account impersonation denial, the signed-in Google account needs roles/iam.serviceAccountTokenCreator on the configured service account. This is an IAM issue, not a Uvicorn issue.

15. Useful checks

Check distinct machines:

sql
SELECT DISTINCT machine_name
FROM market-lens-506611.manufacturing_rag.document_chunks
ORDER BY machine_name;

Check embedding compatibility:

sql
SELECT machine_name, embedding_model, embedding_dimensions, COUNT(*) AS chunks
FROM market-lens-506611.manufacturing_rag.document_chunks
GROUP BY machine_name, embedding_model, embedding_dimensions
ORDER BY machine_name;

Every row used in shared retrieval should report gemini-embedding-001 and 768.

Run backend tests:

powershell
cd backend
.\.venv\Scripts\python.exe -m unittest discover -s tests -v

Run frontend checks:

powershell
cd frontend
npm.cmd run lint
npm.cmd run build

16. What is currently implemented

- Multiple machine manuals in one document_chunks table.
- Machine-independent retrieval.
- 768-dimensional Gemini embeddings.
- Semantic cosine search plus keyword scoring.
- Top semantic seeds followed by same-page context expansion.
- Text, table, image, PDF-page, video, and QR-link support.
- Image descriptions in the final answer.
- Evidence validation that rejects unknown or invented citations.
- Session history and guided troubleshooting state.
- CORS, upload validation, bounded timeouts, and service error responses.
- Offline Document AI QR extraction and verified QR-link storage.

- Ten offline backend tests covering API behavior, media completeness, evidence validation, and QR provenance.

The safest mental model is: BigQuery finds approved evidence, Gemini explains only that evidence, and the response builder decides what can be shown to the user.